

Apuntes de C++

Luciano Arellano

September 22, 2026

Contents

1	Sintaxis basica	2
1.1	hello world	2
1.2	Compilacion	2
1.3	Declaracion, tipos y <i>casting</i>	3
1.4	Loops, if else, etc.	4
2	Funciones	5
2.1	Desafio	6
2.1.1	<i>Skeleton code</i> para el desafio	6
3	Manejo de memoria	7
4	Arreglos y Vectores	8
4.1	Arreglos	8
4.2	Desafio 2: como esta ordenado un array de dos dimensiones?	9
4.3	Strings	9
4.4	Alocacion dinamica de memoria	9
4.5	Vectores	10
5	Programacion orientada al objeto (OOP)	10
5.1	Clases	11
5.1.1	Constructor y destructor	12
6	ROOT	13
6.1	Interprete de C++	13
6.2	Histogramas	13
6.3	TFiles	13
6.4	TTrees	14
6.4.1	Draw()	15
6.4.2	TTreeReader	15
6.4.3	SetBranchAddress	15
6.4.4	TTree::MakeClass	15
6.5	RDataFrame	15
6.6	TGenPhaseSpace	15

1 Sintaxis basica

1.1 hello world

Partimos con un ejemplo. `hello_world.cpp`

```
1 #include <iostream>
2
3 int main() {
4     std::cout << "hello world." << std::endl;
5     return 0;
6 }
```

La primera linea importa un *header* (algo análogo a importar un módulo en Python) que en este caso nos permite sacar input/outputs a pantalla.

Nuestro programa en C++ siempre debe tener una (unica) funcion llamada **main**. La funcion debe retornar un numero entero, y es estandar que retorne 0 si el programa se ejecutó correctamente. El *scope* de la funcion esta delimitado por parentesis llave { }.

Esta funcion main tiene dos *statements*, los cuales en C++ siempre deben terminar en ; (punto coma, o *semicolon*). A diferencia de Python, en C++ no importa la indentación, ni los saltos de linea. El siguiente código también es C++ sintacticamente correcto (por favor no escriban asi su código!):

```
#include <iostream>
int main() {std::cout << "hello world." << std::endl; return 0;}
```

La linea de output (L4) es un poco mas dificil de explicar por ahora, pero podemos imaginar que estamos llamando a una “funcion” `cout` que vive en `std`, como por ejemplo en python podriamos hacer `import numpy as np` y luego `np.sqrt()` para llamar la funcion raiz cuadrado de numpy.

El **namespace** `std` es una coleccion de funciones y objetos. Es comun que la gente escriba

```
1 #include <iostream>
2 using namespace std;
3 int main() {
4     cout << "hello world." << endl;
5     return 0;
6 }
```

En este caso nos ahorramos tipear `std::` cada vez que llamamos a alguien de ese namespace (L4) (que generalmente es bastantes veces para `std`). El analogo en Python seria `import numpy` a secas. Pero si hacemos esto perdemos de vista donde vive cada sujeto, asi que no lo haremos.

Uno puede crear sus propios namespaces, pero eso va más allá del enfoque de nuestro curso.

1.2 Compilacion

C++ es un lenguaje compilado. Escribimos primero nuestro archivo de texto `hello_world.cpp` y luego compilamos. Aqui “cpp” viene de “C plus plus”, las extensiones usadas son convenciones, pero mejor seguir con `cpp`. Otras usadas son `.cxx` (signos + diagonales), `.cc`

(usada en windows?) y `.C` (no usar, es mas para C).

```
g++ hello_world.cpp
```

esto por defecto va a generar un ejecutable llamado `a.out` que podemos ejecutar con

```
./a.out
```

Si queremos darle un nombre usamos el flag de output `-o`

```
g++ hello_world.cpp -o hello
```

que ejecutamos como

```
./hello
```

Si hay un error de sintaxis, muchas veces el compilador nos lo va a decir, pero tambien puede haber *warnings* que, aunque no fatales, nos pueden advertir de algun peligro. A esta altura, nos conviene que el compilador nos muestre todos los warnings con `-Wall`

```
g++ hello_world.cpp -o hello -Wall
```

Si queremos realmente tener un codigo seguro, podemos poner al compilador con el maximo nivel de exigencia con

```
g++ hello_world.cpp -o hello -Wall -pedantic-errors
```

1.3 Declaracion, tipos y *casting*

C++ es un lenguaje *estrictamente tipeado* (strongly typed), es decir que cada variable debe tener un tipo bien definido e inmutable. Uno tiene que **declarar** una variable para reservar el espacio de memoria que va a usar, y luego se le **asigna** un valor. En contraste, en *Python* las variables no se declaran.

```
1 int a; // declaramos "a" como un entero
2 a = 18; // le asignamos el valor 18
```

Se puede tambien declarar e **inicializar** en un solo statement. Un par de formas comunes son:

```
1 int a = 18; // muy comun de ver
2 int b {18}; // forma moderna (desde C++11)
3 float a {18.0}; // ERROR! "a" ya fue declarado como int
```

En estos casos inicializar es casi lo mismo que asignar, pero esto no es siempre el caso. Uno puede tener una rutina de inicializacion mas compleja.

Uno puede definir una variable como constante, lo que impide que su valor sea reasignado. En este caso debe ir inicializada.

```
1 const float pi {3.1415926}; // la variable pi queda fija
2 pi = 3.0; // ERROR! no se puede asignar un const
3 const float b; // Warning o error? (revisen!) const no inicializada
4 #define PI 3.14 // old school. No usar esta forma.
```

Si asignamos un valor de un tipo a una variable de distinto tipo necesitamos convertirlo, o hacer un *casting*. El compilador puede a veces convier tipos sin avisarnos: eso un casting implícito.

```
1 int myint {4};
2 float myfloat = myint; // ?? cast de int a float?
3
4 float myfloat2 {13.8};
5 int myint2 = myfloat; // ?? float a int? prueben!
```

Para tipos simples el compilador generalmente hace un buen trabajo, pero ustedes deberían querer tener control total sobre esto.

```
1 double x {18.6};
2 int n = (int) x; // cast estilo C
3 int m = static_cast<int>(x); // estilo C++, conversion explicita
```

El error mas tipico es la division enetra. Si uno divide dos int o dos float, C++ usa funciones “division” distintas

```
1 int ai {1};
2 int bi {2};
3 std::cout << ai/bi << std::endl;
4
5 float af {1.0};
6 float bf {2.0};
7 std::cout << af/bf << std::endl;
8
9 // que sucedera con ai/bf? af/bi?
```

A este nivel parece muy sencillo, pero despues vamos a tener cosas mas delicadas

```
1 Animal *a = ...;
2 Dog *d = static_cast<Dog*>(a);
```

C++ nos da control absoluto de como hacer castings. Otra ventaja, podemos hacer mucho mas facilmente un Ctrl+f a todas las instancias de `static_cast`.

1.4 Loops, if else, etc.

```
1 // i++ es lo mismo que i+=1
2 for ( int i {0}; i < 10; i++ ) {
3     std::cout << i << std::endl;
4 }
5
6 int b {10};
7 while ( b > 0; b-- ) {
8     std::cout << b << std::endl;
9 }
10
11 int n {10};
12 if ( 20 > n ) {
13     std::cout << "menor que 20" << std::endl;
14 } else if ( n > 0 ) {
15     std::cout << "mayor que 0, menor que 20" << std::endl;
16 } else {
17     std::cout << "menor o igual que 0 o mayor o igual que 20" << std::endl;
```

18 }

2 Funciones

Las funciones en C++ se definen como

```
1 // esta funcion debe retornar un int
2 int mi_funcion_int() {
3     return 1;
4 }
5
6 // funcion tipo void no retorna nada!
7 void funcion_void(int arg1) { // toma como argumento un int
8     std::cout << "El entero es: " << arg1 << std::endl;
9 }
```

Uno puede declarar una funcion antes del main, e **implementarla** despues del main():

```
1 void myfunc(int a);
2
3 int main() {
4     int a {78};
5     myfunc(a);
6 }
7
8 void myfunc(int a) {
9     std::cout << "a vale " << a << std::endl;
10 }
```

Como mencionamos la clase anterior, uno puede **sobrecargar** funciones. Tenemos una funcion que eleva al cuadrado, podriamos usar distintas funciones para elevar int y floats

```
1 int cuadrado_i(int a) {
2     return a*a;
3 }
4
5 float cuadrado_f(float a) {
6     return a*a;
7 }
```

pero tiene mucho mas sentido que uno haga una pura funcion, que es elevar al cuadrado.

```
1 int cuadrado(int a) {
2     return a*a;
3 }
4
5 float cuadrado(float a) {
6     return a*a;
7 }
```

2.1 Desafio

Sobrecargue la funcion `cuadrado` para que calcule el cuadrado de una matriz 2×2 . La funcion debe retornar tipo `void` y sacar por pantalla los digitos en orden correcto.

Ejecute la funcion para las matrices

- $m_1 = \begin{pmatrix} 2 & 3 \\ 4 & 5 \end{pmatrix}$
- $m_2 = \begin{pmatrix} -1 & 0 \\ -2 & 1 \end{pmatrix}$
- $m_3 = \begin{pmatrix} 2.1 & 0.7 \\ -7.2 & 0.6 \end{pmatrix}$

Ejemplo de output para $m_I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$

```
Cuadrado de m_I:  
1 0  
0 1
```

2.1.1 *Skeleton code* para el desafio

```
1  #include <iostream>  
2  
3  // int  
4  int cuadrado(int a) {  
5      std::cout << "Llamaste a la funcion int\n";  
6      return a*a;  
7  }  
8  
9  // float  
10 float cuadrado(float a) {  
11     std::cout << "Llamaste a la funcion float\n";  
12     return a*a;  
13 }  
14  
15 // matrix  
16 /*  
17  Implementacion de la funcion:  
18  deben definir cuantos argumentos va a tener  
19 */  
20 void cuadrado(){  
21     std::cout << "Llamaste a la funcion matrix\n";  
22 }  
23  
24 int main() {  
25     int a {9};  
26     std::cout << cuadrado(a) << std::endl;  
27     float b {3.2};  
28     std::cout << cuadrado(b) << std::endl;  
29     // cuadrado(matrix??);  
30     return 0;  
31 }
```

3 Manejo de memoria

En la clase anterior hemos aprendido el rol de la memoria en un computador. Ahora vemos que en C++ tenemos que declarar cada variable, dejando fijo su tipo. Al inicializar una variable, la direccion de memoria que ocupa queda fija. Podemos acceder a ella con el operador &

```
1 int a {10};
2 std::cout << "El valor de a es " << a << std::endl;
3 std::cout << "La direccion de memoria de a es " << &a << std::endl;
```

El valor de a es 10

La direccion de memoria de a es 0x7fffd31e8844

Noten que con cada ejecucion la direccion de memoria cambia. Realmente nos estamos metiendo en el cerebro del computador.

Podemos declarar una variable como **referencia** a otra,

```
1 int a {10};
2 int &r {a}; // r es una referencia a "a"
3 r = 30;     // r ES a
4 std::cout << "El valor de a es " << a << std::endl;
```

Esto es particularmente util para funciones, si queremos que una funcion cambie nuestros numeritos, le pasamos valores por referencia

```
1 int by_value( int a ) { // hace una copia de la variable que le pasamos
2     a = a+10; // le suma 10 a la copia
3     return a; // devuelve la copia
4 }
5
6 void by_reference (int &a ) { // le pasamos la variable de verdad
7     a = a+10; // nos cambia la variable original
8 }
9
10 int main () {
11     int a {10};
12     int b;
13     b = by_value(a);
14     std::cout << "a, b: " << a << " " << b << std::endl;
15     by_reference(a);
16     std::cout << "a: " << a << std::endl;
17 }
```

Esto es tremendamente poderoso! una vez que sabemos la direccion de memoria, podemos trabajar directamente con ella a traves de un **puntero** (*pointer*).

```
1 int *b; // b es un puntero de enteros
2 std::cout << "El valor del puntero b es " << b << std::endl;
3 b = &a;
4 std::cout << "El valor del puntero b es " << b << std::endl;
5 std::cout << "b apunta al valor " << *b << std::endl;
```

El valor del puntero b es 0x7f512834548f

El valor del puntero b es 0x7fffd31e8844

b apunta al valor 10

Notemos que inicialmente `b` apunta a cualquier lado. Pero al asignar `b = &a` ahora apunta a la direccion de memoria de `a`. Y entonces podemos *dereferenciar* `b` usando `*b` para obtener el valor `int` guardado en memoria. No solo podemos obtener el valor en memoria, podemos cambiarlo. `a` no es dueño de la memoria, nosotros tenemos control total

```

1  int a {10};
2  std::cout << "El valor de a es " << a << std::endl;
3  int *b {&a}; // b es un puntero hacia a
4  *b = 20;      // el valor hacia el cual b apuntaba ahora es 20
5  std::cout << "El valor de a es " << a << std::endl;

```

Cuidemos el lenguaje con la siguiente tabla:

Usado en \ operador	<code>&</code>	<code>*</code>
Declaracion	Referencia	Puntero
Expresion	Direccion de memoria	Dereferencia

4 Arreglos y Vectores

4.1 Arreglos

Como vimos la clase anterior, un arreglo es una estructura de datos basica tremendamente eficiente (lectura $\mathcal{O}(1)$) pero que tiene un tamaño fijo. Se reservan N bloques de memoria contiguos en el momento de declaracion,

```

1  int a[3];
2  a[0] = 1;
3  a[1] = 2;
4  a[2] = -1;
5
6  float b[2] = { -2.3, 4.5 };
7
8  for ( int i = 0; i < 3; i++ ) {
9      std::cout << "a[" << i << "]: " << a[i] << std::endl;
10     std::cout << "b[" << i << "]: " << b[i] << std::endl;
11     // notemos que b sale del rango
12 }

```

En este ejemplo definimos `b` con largo 2 pero le pedimos el tercer elemento. El compilador no se preocupa de esto ya que no le estamos pidiendo algo invalido. Esta falta de checks es un arma de doble filo. Es lo que permite que los arreglos en C++ sean extremadamente eficientes, tener pocos chequeos.

Un array es realmente un puntero. Si un float son 4 bytes, el arreglo lo unico que fija es la primera direccion de memoria, y luego va sumando 4 cada vez. Supongamos que la primera direccion es `&b[0] → 1320`, entonces `&b[1] → 1324`, y en general `&b[n] → 1320 + 4 * n`. Veamoslo explicitamente

```

1  const int nentries {5};
2  float array[nentries];
3  for ( int i {0}; i < nentries; i++ ) {
4      array[i] = 1.1*i;
5      std::cout << i << " " << array[i] << " " << *(array+i) << std::endl;
6  }

```

vemos que `*array` apunta a la primera direccion de memoria, luego la i -esima direccion se obtiene con `*(array+i)`.

4.2 Desafio 2: como esta ordenado un array de dos dimensiones?

```
1  int a[2][3]; // ??
```

4.3 Strings

Un `string` es una cadena de texto (en general). En C++ se implementa como un arreglo de caracteres. Vimos que los caracteres son la unidad basica, cada uno ocupa un byte en memoria. No vamos a ahondar mucho con strings en C++ porque despues en ROOT vamos a tener mejores maneras de trabajar con strings.

```
1  #include <iostream>
2  #include <string>
3
4  int main() {
5
6      char a {'b'};
7      char c {'c'};
8      std::string mystring{"hello world"};
9
10     for ( int i {0}; i < mystring.length(); i++ ) {
11         std::cout << i << " " << mystring[i] << std::endl;
12     }
13     return 0;
14 }
```

4.4 Alocacion dinamica de memoria

Uno debe definir el tamano de los arreglos al momento de declarar, pero no siempre lo sabemos. Primero, veamos como generar numeros aleatorios.

```
1  #include <iostream>
2  #include <random>
3
4  int main() {
5      std::mt19937 gen {2399}; // semilla 2399
6      std::uniform_int_distribution<int> dist {0, 10};
7      for ( int i {0}; i < 15; i++ ) {
8          int nparticles { dist(gen) };
9          std::cout << nparticles << std::endl;
10     }
11     return 0;
12 }
```

Aqui usamos `mt19937` (generador Mersenne Twister) para generar numeros pseudo-aleatorios a partir de una semilla. Tenemos una distribucion uniforme de enteros entre 0 y 10. La funcion `dist(gen)` nos entrega los numeros aleatorios. Supongamos ahora que en cada evento tenemos una cantidad `dist(gen)` de particulas (`int nparticles { dist(gen) }`) y queremos hacer un arreglo con las energias de todas las particulas en cada evento. Si tratamos de hacer un arreglo poniendo `nparticles`, el compilador nos deja, pero

4.5 Vectores

Los vectores son la implementación de la estructura de datos de arreglos dinámicos, es decir son arreglos, pero podemos agregar elementos, luego de lo cual se reserva un nuevo bloque de memoria contiguo que acomode el nuevo número.

Los vectores son tremendamente útiles en física de partículas, dado que no sabemos a priori el número de partículas que habrá en nuestro evento, o número de partículas que pasen nuestra selección, por lo que necesitamos algo de dinamismo. Es una de las herramientas de C++ base que serán más usadas cuando pasemos a ROOT.

```
1  #include <iostream>
2  #include <vector>
3
4  int main() {
5      std::vector<int> v1;           // vector de int (vacío)
6      std::vector<float> v2 { 3.3, 1/34.}; // vector de floats (2 inicialmente)
7      v1.push_back(4);             // le agregamos un elemento a v1 (al final)
8
9      for ( int i {0}; i < v1.size(); i++ ) {
10         std::cout << "v1[" << i << "]: " << v1[i] << std::endl;
11     }
12
13     for ( int i {0}; i < v2.size(); i++ ) {
14         std::cout << "v2[" << i << "]: " << v2[i] << std::endl;
15     }
16     return 0;
17 }
```

Notemos que en la declaración de los vectores aparece el tipo que contiene encerrado entre paréntesis angulares: `<int>`, `<float>`, etc. Esto es algo que no habíamos visto antes. Los `std::vector` son estructuras llamadas **templates**, que les permiten contener distintos tipos sin alterar la sintaxis de uso. Los templates son una herramienta muy poderosa, podemos hacer templates de funciones y de clases, pero eso está fuera del enfoque de nuestro curso. Los vectores son quizás el único ejemplo de templates que veremos.

5 Programación orientada al objeto (OOP)

Hemos visto que podemos usar funciones para manipular datos, sin embargo los datos no viven en el vacío, por ejemplo, pueden estar correlacionados entre sí. Consideremos una partícula a altas energías. El momentum (p_x, p_y, p_z), la masa m , y energía E están relacionados mediante

$$E^2 = m^2 + p_x^2 + p_y^2 + p_z^2, \quad (1)$$

donde m es un *invariante relativista*, es decir, independiente del sistema de referencia, y depende exclusivamente del tipo de partícula (muon, proton, etc.), por lo que si escribimos una función que cambie la energía y queremos que se mantenga esta relación fundamental, **tenemos** que cambiar el momentum.

El lenguaje C tiene **structs** (estructuras), esto era la mejor solución en aquellos tiempos. Por ejemplo, uno define una partícula y le cambia el momentum usando una función

```

1 // struct tipo C
2 struct Particula {
3     float energy;
4     float mass;
5     float momentum[3];
6 };
7
8 Particula muon; // declaramos un muon
9 muon.mass = 0.1057; // fijamos la masa del muon
10 muon.energy = 10.0; // le damos energia 10 GeV
11 // momentum no puede ser cualquier cosa si queremos relacion energia-momentum
12
13 // esta funcion NO chequea energia-momentum y cambia los valores de p
14 void set_momentum(Particula &p, float px, float py, float pz) {
15     p.momentum[0] = px;
16     p.momentum[1] = py;
17     p.momentum[2] = pz;
18 }
19 set_momentum(muon, 30.0, 0.0, 0.0);
20 // ahora  $10^2 \neq 0.1057^2 + 30^2$ , mal!!
21
22 muon.energy = 999.0; // mas grosero aun!!

```

Los `structs` siguen existiendo en C++ , pero tienen muchas debilidades. Cualquier programador puede agarrar nuestra `struct Particula` y escribir sus propias funciones que no respeten las leyes de la física. Eso es porque cualquiera puede acceder directamente a la energía, masa y momentum y cambiar los valores libremente. Esto es un problema que los objetos y las clases vienen a resolver.

5.1 Clases

En 1979, Bjarne Stroustrup comienza a expandir C a “C con clases”, lo que eventualmente lo llevaría a la creación de C++. Una **clase** es algo más avanzado que una **struct**. Tiene variables, funciones, y reglas que definen de qué manera y a quienes se puede acceder desde fuera de ella.

```

1 class Particula {
2 private:
3     float energy;
4     float mass;
5     float momentum[3];
6     void adjust_energy() {};
7 public:
8     void set_momentum(float, float, float); // funcion declarada
9 };
10
11 // implementacion de la funcion declarada
12 void Particula::set_momentum(float px, float py, float pz)
13 {
14     // seteamos las variables
15     momentum[0] = px;
16     momentum[1] = py;
17     momentum[2] = pz;
18     adjust_energy(); // ajustamos la energia si es necesario
19 }

```

En este ejemplo definimos una clase llamada `Particula` que tiene **variables** y

funciones miembros, privadas y públicas (keywords `private` y `public`). No se pueden modificar o acceder a los valores privados salvo con lo que está definido en lo público. De esta manera, controlamos de qué manera se accede a la información. En este caso, la energía, masa y momentum son privados, nadie los toca. La única manera de cambiar el momentum es con la función `set_momentum()` que hemos definido como pública.

Las **clases** son estructuras que nos permiten agrupar **objetos** que tienen características en común, y establecer las formas en que interactúan entre ellos. Este nivel de abstracción nos va a permitir hacer cosas más poderosas que simplemente funciones. En este ejemplo además damos un *hint* de cómo podríamos solucionar nuestro problema original. Al setear el momentum, podemos llamar a una función `adjust_energy()` (no implementada) que ajuste la energía para que se mantenga la relación energía momentum.

Podemos ver ahora que un **struct** es como una clase donde todos los miembros son public. Como regla general, queremos que las variables miembro sean privadas y acceder a ellas y modificarlas usando metodos publicos (*setters* y *getters*).

```
1 class Particula {
2 private:
3     float energy;
4 public:
5     void set_energy(float e) { energy = e; } // cambiamos el valor de energia (setter)
6     float get_energy() { return energy; }    // accedemos al valor de energia (getter)
7 };
```

5.1.1 Constructor y destructor

Dos funciones especiales que generalmente queremos incluir son el constructor y destructor. Por ahora vamos a ver solo el constructor, y veremos el destructor más adelante. El uso más básico que se le puede dar a un constructor es asignar valores iniciales a las variables miembro, aunque uno puede hacer constructores más complicados.

```
1 #include <iostream>
2 #include <cmath>
3
4 class Particula {
5 private:
6     float energy;
7     float mass;
8     float momentum[3];
9 public:
10    Particula() {} // constructor por defecto
11    ~Particula() {} // destructor por defecto
12    Particula(float m) {
13        // constructor 1 argumento (masa)
14        mass = m;
15    }
16    Particula(float e, float px, float py, float pz) {
17        // constructor energia-momentum
18        // calcula y fija la masa
19        energy = e;
20        momentum[0] = px;
21        momentum[1] = py;
22        momentum[2] = pz;
```

```

23     mass = std::sqrt(e*e - px*px - py*py - pz*pz);
24 }
25 void print_mass() { std::cout << "masa " << mass << std::endl; }
26 };
27
28 int main()
29 {
30     Particula p1;           // constructor por defecto
31     Particula p2(0.1057); // constructor con masa
32
33     // constructor energia-momentum
34     Particula p3(10.0,      // e
35                 0.0,        // px
36                 0.0,        // py
37                 9.999441); // pz
38
39     p1.print_mass();
40     p2.print_mass();
41     p3.print_mass();
42
43     return 0;
44 }

```

En este ejemplo, `p1`, `p2` y `p3` son objetos (o instancias) de la clase `Particula`. El programa (compilador) sabe cual constructor usar basado en el número y tipo de argumentos. A diferencia de Python tenemos que ser consistentes siempre con el orden de los argumentos, en el tercer constructor *no podemos* pasarlos en desorden usando *key-word arguments* como `Particula p3(py=0.0, e=10.0, pz=9.999441, px=0.0)`. En C++ los argumentos se pasan sólo por posición.

Al momento de crear una instancia de `Particula` (llamar al constructor), el programa reserva la memoria necesaria para contener el objeto completo (variables metodo). Al llamar al destructor, se libera la memoria.

6 ROOT

6.1 Interprete de C++

Abrimos ROOT y podemos hacer algunos calculos

```

2*Math::Pi()

sqrt(3) + exp(-1)

```

6.2 Histogramas

```

TH1F("h1", "Gaussian;x;y", 100, -5, 5)

h1.FillRandom("gaus", 10000)

h1.Draw()

```

6.3 TFiles

Podemos acceder a datos usando archivos ROOT. Estos pueden contener practicamente cualquier objeto de ROOT, pero los más importantes van a ser `TTree`, histogramas, y

TGraphs.

```
1 TFile f("events_1000.root");           // abre el archivo "events_1000.root"
2 TFile f_out("salida.root", "recreate"); // crea un archivo de salida "salida.root".
```

En el archivo de salida del ejemplo, la opción `recreate` sobrescribe el archivo cada vez que se ejecuta el programa. En ROOT siempre estamos "dentro de un archivo". En una sesión interactiva, podemos ver el contenido usando `.ls`.

```
root [0] TFile f("events_1000.root");           // abre el archivo "events_1000.root"
root [1] .ls
TFile** events_1000.root
TFile*  events_1000.root
KEY: TTree t;2 Eventos de Pythia8 [current cycle]
KEY: TTree t;1 Eventos de Pythia8 [backup cycle]
KEY: TH1F h_wide;1
KEY: TH1F h_zoom;1
root [2] TFile f_out("salida.root", "recreate"); // crea un archivo de salida "salida.root"
root [3] .ls
TFile** salida.root
TFile*  salida.root
root [4] f.cd()
```

6.4 TTrees

La manera más común de almacenar datos en ROOT es usando **TTrees**, que usan estructura de árbol. Los árboles tienen **TBranches** los cuales a su vez pueden tener **TLeaves**. Estos pueden contener cualquier tipo de datos, `int`, `float`, `std::vector<Particula>`, etc.

En ROOT interactivo podemos ver lo que contiene un árbol con los métodos `Print()` y `Scan()`

```
root [0] TFile f("events_1000.root")
root [1] t->Print()
*****
*Tree      :t          : Eventos de Pythia8                                     *
*Entries   :    1000   : Total =          42017892 bytes  File Size =    33309375 *
*          :          : Tree compression factor =    1.26                       *
*****
*Br    0 :phi          : vector<double>                                         *
*Entries :    1000   : Total Size=    3819842 bytes  File Size =    3693596 *
*Baskets  :    115   : Basket Size=    3207680 bytes  Compression=    1.03    *
*.....*
*Br    1 :eta          : vector<double>                                         *
*Entries :    1000   : Total Size=    3819842 bytes  File Size =    3701259 *
*Baskets  :    115   : Basket Size=    3207680 bytes  Compression=    1.03    *
*.....*
*Br    2 :y           : vector<double>                                         *
*Entries :    1000   : Total Size=    3819604 bytes  File Size =    3701194 *
*Baskets  :    115   : Basket Size=    3207168 bytes  Compression=    1.03    *
```

```

*.....*
*Br    3 :pT          : vector<double>                                *
*Entries :      1000 : Total  Size=    3819723 bytes  File Size  =    3615218 *
*Baskets :      115 : Basket Size=    3207680 bytes  Compression=    1.06   *
*.....*
*Br    4 :E           : vector<double>                                *
*Entries :      1000 : Total  Size=    3819604 bytes  File Size  =    3658345 *
*Baskets :      115 : Basket Size=    3207168 bytes  Compression=    1.04   *
*.....*
*Br    5 :px          : vector<double>                                *
*Entries :      1000 : Total  Size=    3819723 bytes  File Size  =    3660037 *
*Baskets :      115 : Basket Size=    3207680 bytes  Compression=    1.04   *
*.....*
*Br    6 :py          : vector<double>                                *
*Entries :      1000 : Total  Size=    3819723 bytes  File Size  =    3660358 *
*Baskets :      115 : Basket Size=    3207680 bytes  Compression=    1.04   *
*.....*
*Br    7 :pz          : vector<double>                                *
*Entries :      1000 : Total  Size=    3819723 bytes  File Size  =    3706720 *
*Baskets :      115 : Basket Size=    3207680 bytes  Compression=    1.03   *
*.....*
*Br    8 :vx          : vector<double>                                *
*Entries :      1000 : Total  Size=    3819723 bytes  File Size  =    1304922 *
*Baskets :      115 : Basket Size=    3207680 bytes  Compression=    2.93   *
*.....*
*Br    9 :vy          : vector<double>                                *
*Entries :      1000 : Total  Size=    3819723 bytes  File Size  =    1304770 *
*Baskets :      115 : Basket Size=    3207680 bytes  Compression=    2.93   *
*.....*
*Br   10 :vz          : vector<double>                                *
*Entries :      1000 : Total  Size=    3819723 bytes  File Size  =    1292379 *
*Baskets :      115 : Basket Size=    3207680 bytes  Compression=    2.95   *
*.....*

```

Hay varias maneras de leer un arbol.

6.4.1 Draw()

6.4.2 TTreeReader

6.4.3 SetBranchAddress

6.4.4 TTree::MakeClass

6.5 RDataFrame

6.6 TGenPhaseSpace

```

root [0] TFile f("events_1000.root")
(TFile &) Name: events_1000.root Title:
root [1] t->Scan()
*****
*      Row      * Instance *      phi *      eta *      y *      pT *      E *      px *      py *      pz *
*****
*      0 *      0 * 2.5625868 * 9.1413071 * 8.9572623 * 0.2092309 * 976.37166 * -0.175127 * 0.1144894 * 976.37162 *
*      0 *      1 * 1.7242741 * 9.8225445 * 9.1204624 * 0.5360556 * 4943.7533 * -0.081950 * 0.5297545 * 4943.7532 *
*      0 *      2 * -1.459065 * 7.6889850 * 7.6438709 * 0.4542041 * 496.02664 * 0.0506430 * -0.451372 * 496.02641 *
*      0 *      3 * -2.068258 * -0.954570 * -0.934656 * 0.5924349 * 0.8944344 * -0.282708 * -0.520629 * -0.655403 *
*      0 *      4 * 0.0931196 * -1.693564 * -0.837655 * 0.1889297 * 0.7251463 * 0.1881111 * 0.0175676 * -0.496409 *
*      0 *      5 * 2.3552890 * -0.725177 * -0.695407 * 1.5428108 * 2.0275911 * -1.089943 * 1.0919193 * -1.219483 *
*      0 *      6 * 1.9113555 * -1.348590 * -1.001723 * 0.8149615 * 1.9202358 * -0.272208 * 0.7681566 * -1.463828 *
*      0 *      7 * -3.097149 * -2.363283 * -2.354394 * 1.0327454 * 5.5372214 * -1.031725 * -0.045882 * -5.438269 *
*      0 *      8 * 0.8527377 * -5.602759 * -5.427479 * 0.2153928 * 29.205151 * 0.1417120 * 0.1622089 * -29.20402 *
*      0 *      9 * 0.9094210 * -6.227144 * -6.204060 * 0.6420792 * 162.54553 * 0.3943668 * 0.5066956 * -162.5442 *
*      0 *     10 * -0.586850 * -5.216973 * -5.182482 * 0.5222472 * 48.146369 * 0.4348695 * -0.289189 * -48.14333 *
*      0 *     11 * 0.5575707 * -0.400346 * -0.395333 * 0.8513862 * 0.9310520 * 0.7224378 * 0.4504909 * -0.350028 *
*      0 *     12 * -0.993196 * -0.482774 * -0.450469 * 0.3489172 * 0.4145745 * 0.1905138 * -0.292314 * -0.175068 *
*      0 *     13 * -2.150573 * -0.473952 * -0.370417 * 0.1686377 * 0.2340931 * -0.092386 * -0.141079 * -0.082952 *
*      0 *     14 * 3.036034 * -0.225641 * -0.217226 * 0.4924781 * 0.5239980 * -0.489736 * 0.0518888 * -0.112069 *
*      0 *     15 * 1.0777579 * 2.7331575 * 2.7071749 * 0.6016381 * 4.6486651 * 0.2847581 * 0.5299823 * 4.6074547 *
*      0 *     16 * 2.9955246 * 2.8594369 * 2.7493380 * 0.2800667 * 2.4558136 * -0.277084 * 0.0407634 * 2.4357962 *
*      0 *     17 * 2.3553125 * 3.3187136 * 3.3174694 * 2.7925407 * 38.622473 * -1.972882 * 1.9763653 * 38.521133 *
*      0 *     18 * 0.1489638 * 1.8956601 * 1.8080405 * 0.3104323 * 1.0657600 * 0.3069944 * 0.0460723 * 1.0099486 *
*      0 *     19 * -1.531362 * 0.6558470 * 0.6226606 * 0.3950473 * 0.5028559 * 0.0155740 * -0.394740 * 0.2780681 *
*      0 *     20 * 2.0986925 * 0.9543868 * 0.7515093 * 1.0487231 * 1.8236080 * -0.528259 * 0.9059590 * 1.1599033 *
*      0 *     21 * 1.5981764 * 0.3443279 * 0.2603547 * 1.0632221 * 1.4663561 * -0.029107 * 1.0628236 * 0.3733743 *
*      0 *     22 * 2.1670013 * -0.318793 * -0.258370 * 0.6717377 * 0.8616174 * -0.377184 * 0.5558444 * -0.217791 *
*      0 *     23 * -0.279999 * 4.9768063 * 4.3166536 * 0.2979418 * 21.609016 * 0.2863386 * -0.082337 * 21.601321 *
*      0 *     24 * -0.288571 * 1.3574480 * 1.0718607 * 0.4984806 * 1.1446774 * 0.4778691 * -0.141858 * 0.9044797 *
Type <CR> to continue or q to quit ==>

```